

SAS® Macro

Macro Variable

RangTech
Enhancing the way IT works

In this lesson, you learn to

- recognize some benefits of using macro variables
- substitute the value of a macro variable anywhere in a program
- identify and display automatic macro variables
- create and display user-defined macro variables
- recognize how macro variables are processed
- display macro variable values and other text in the SAS log
- use macro character functions
- combine macro references with adjacent text or with other macro variable references
- use macro quoting functions.

```
%let year=2002;
proc print data=sasuser.schedule;
where year(begin_date)=&year;
title "Scheduled Classes for &year"; run;
proc means data=sasuser.all sum;
where year(begin_date)=&year;
class location; var fee;
title1 "Total Fees for &year Classes";
title2 "by Training Center";
run;
```

RangTech
Enhancing the way IT works

- **SAS Macro facility**

  A tool for extending and customizing SAS and for reducing the amount of program code you must enter in order to perform common tasks.

Two types of macro variables:

- **automatic macro variables**, which are provided by SAS
- **user-defined macro variables**, whose values you create and define.

- A macro variable reference is created by preceding the macro variable name with an ampersand (&).

Ranglech
Enhancing the way IT works

Once SAS code is in the input stack, SAS

- reads the text in the input stack (left-to-right, top-to-bottom)
- routes text to the appropriate compiler upon demand
- suspends this activity when a step boundary such as a RUN statement is reached
- executes the compiled code if there are no compilation errors
- repeats this process for any subsequent steps.

Automatic Macro Variables

Name	Value
SYSDATE	the date of the SAS invocation (DATE7.)
SYSDATE9	the date of the SAS invocation (DATE9.)
SYSDAY	the day of the week of the SAS invocation
SYSTIME	the time of the SAS invocation
SYSENV	FORE (interactive execution) or BACK (noninteractive or batch execution)
SYSSCP	an abbreviation for the operating system that is being used, such as OpenVMS, WIN, HP 300
SYSVER	the release of SAS that is being used
SYSJOBID	an identifier for the current SAS session or for the current batch job (the user ID or job name for mainframe systems, the process ID (PID) for other systems)

Fixed-Value Automatic Macro Variables

Some automatic macro variables have values that automatically change based on submitted SAS statements.

Name	Value
SYSLAST	the name of the most recently created SAS data set, in the form LIBREF.NAME. This value is always stored in all capital letters. If no data set has been created, the value is _NULL_
SYSPARM	text that is specified when SAS is invoked
SYSERR	contains a return code status that is set by the DATA step and some SAS procedures to indicate if the step or procedure executed successfully

Macro variable, cont.

If the next data set is HOSPITAL, you would replace LAB in three lines of SAS code. The repeated text is repetitious, but necessary. The repetition of the task, changing three lines of code, is not necessary if you use a macro variable as a shortcut to represent the data set name...

Example 1...simple macro variable

```
%let dsn=LAB;
title "DATA SET &dsn";
proc contents data=&dsn;
run;
proc print data=&dsn(obs=10);
run;
```

RangTech
Enhancing the way IT works

%Let

%LET *variable* = *value*;

where

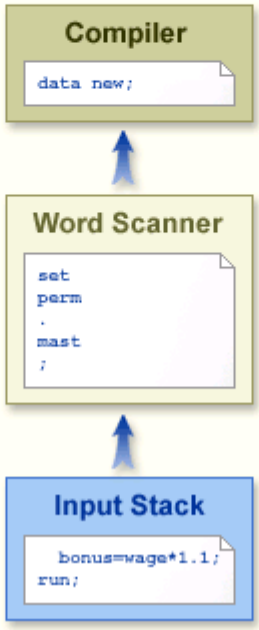
- *variable* is any name that follows the SAS naming convention.
- *value* can be any string from 0 to 65,534 characters.
- if either *variable* or *value* contains a reference to another macro variable (such as &macvar), the reference is evaluated before the assignment is made.
- If *variable* already exists, *value* replaces the current value.

%LET Statement	Variable Name	Variable Value	Length
%let name= Ed Norton ;	name	*Ed Norton*	9
%let name2=' Ed Norton ';	name2	*' Ed Norton '*	13
%let title="Joan's Report";	title	*"Joan's Report"*	15
%let start=;	start		0
%let total=0;	total	*0*	1
%let sum=4+3;	sum	*4+3*	3
%let total=&total+∑	total	*0+4+3*	5
%let x=varlist;	x	*varlist*	7
%let &x=name age height;	varlist	*name age height*	15

Tokenization

Between the input stack and the compiler, SAS programs are **tokenized** into smaller pieces. A component of SAS known as the **word scanner** divides program text into fundamental units called **tokens**.

- Tokens are passed on demand to the compiler.
- The compiler requests tokens until it receives a semicolon.
- The compiler performs a syntax check on the statement.



- A **literal** token is a string of characters that are treated as a unit. The string is enclosed in either single or double quotation marks.
  Examples: "Any text" 'Any text'

- A **number** token is a string of numerals that can include a period or E-notation (real numbers). Date constants, time constants, datetime constants, and hexadecimal constants are also number tokens.
  Examples: 23 109 '01jan2002'd 5e8 42.7

- A **name** token is a string of characters that begins with a letter or underscore and that continues with underscores, letters, or digits. A period can sometimes be part of a name.
  Examples: infile _n_ item3 univariate dollar10.2

- A **special** token is any character or group of characters that has a reserved meaning to the compiler.
  Examples: * / + - ** ; $ ( ) . & %

- A token ends when the word scanner detects
  - the beginning of another token
  - a blank after a token.
  - The maximum length of any token is 32767 characters.

Examples

- var x1-x10 z ;
  This example contains six tokens: **var x1 - x10 z ;**
- title 'Report for May';
  This example contains three tokens: **title 'Report for May' ;**

RangTech
Enhancing the way IT works

Macro Triggers

- Macro variable references and %LET statements are part of the **macro language**. The macro facility includes a macro processor that is responsible for handling all macro language elements. Certain token sequences, known as **macro triggers**, alert the word scanner that the subsequent code should be sent to the macro processor.

The word scanner recognizes the following token sequences as macro triggers:

- **%** followed immediately by a **name token** (such as **%let**)
- **&** followed immediately by a **name token** (such as **&amt**).

When a macro trigger is detected, the word scanner passes it to the macro processor for evaluation. The macro processor

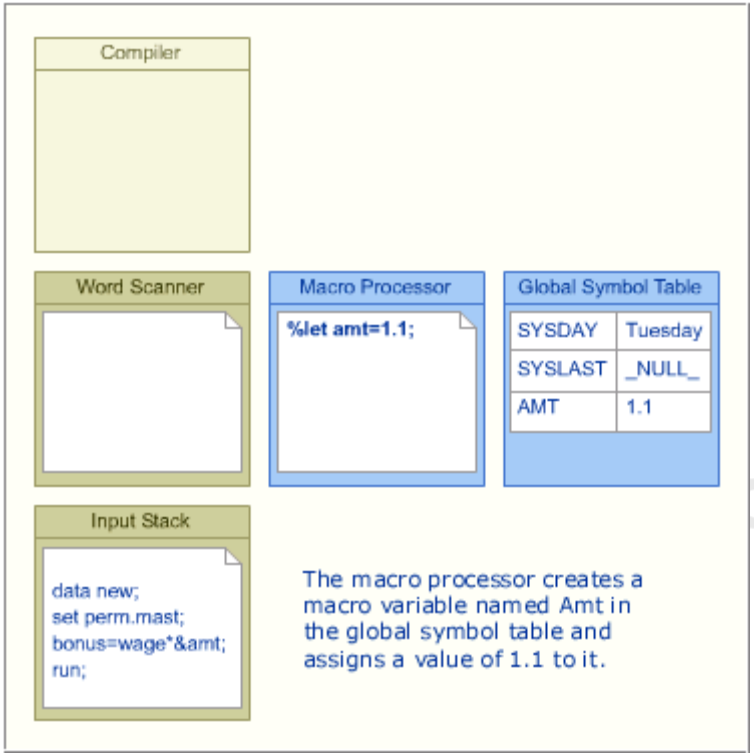
- examines these tokens
- requests additional tokens as necessary
- performs the action indicated.

For macro variables, the processor does one of the following:

- creates a macro variable in the symbol table and assigns a value to the variable
- changes the value of an existing macro variable in the symbol table
- looks up an existing macro variable in the symbol table and returns the variable's value to the input stack in place of the original reference.

The word scanner then resumes processing tokens from the input stack.

RangTech
Enhancing the way IT works


Compiler
Word Scanner
Macro Processor
%let amt=1.1;
Global Symbol Table
SYSDAY
Tuesday
SYSLAST
NULL
AMT
1.1
Input Stack
data new;
set perm.mast;
bonus=wage*&amt;
run;
The macro processor creates a macro variable named Amt in the global symbol table and assigns a value of 1.1 to it.

Macro Processing

1. The input stack contains the following program:

```
%let amt=1.1;
data new;
set perm.mast;
bonus=wage*&amt;      run;
```

2. The word scanner reads the first two tokens of the %LET statement, and it passes that trigger to the macro processor.

3. The macro processor requests additional tokens until it receives a semicolon.

4. The macro processor creates a macro variable named amt in the global symbol table and assigns a value of *1.1* to it.

5. The macro processor gives control back to the word scanner. The word scanner passes the next few statements to the compiler until it reaches the next macro trigger.

6. The word scanner passes &amt to the macro processor.

7. The macro processor looks up amt in the global symbol table and sends the value of amt (*1.1*) back to the input stack.

8. Word scanning continues, but since the reference has been resolved to its value, the word scanner passes the token to the compiler.

9. When SAS detects the RUN statement, which is a step boundary, the step executes.

RongTech
Enhancing the way IT Works

vescontrolbacktothewordscanner.Thewordsca

The %PUT statement

- writes only to the SAS log
- always writes to a new log line, starting in column one
- writes a blank line if text is not specified
- does not require quotation marks around text
- resolves macro triggers in text before text is written
- removes leading and trailing blanks from text unless a macro quoting function is used
- wraps lines when the length of text is greater than the current line size setting
- can be used either inside or outside a macro definition.

```
%put The value of the macro variable CITY is: &city;
```

SAS Log

```
120 %put The value of the macro variable CITY is: &city;
The value of the macro variable CITY is: Dallas
```

RangTech
Enhancing the way IT works

Macro Quoting Functions

The %STR Function

- The %STR function is used to mask (or quote) tokens so that the macro processor does not interpret them as macro-level syntax. Special tokens and mnemonic equivalents include

  ; + - * / , < > = blank LT EQ GT AND OR NOT LE GE NE

The %STR function also

- enables macro triggers to work normally
- preserves leading and trailing blanks in its argument.

Examples

Method One: You could quote all text.

```
%let prog=%str(data new; x=1; run;);
```

Method Two: You could quote only the semicolons.

```
%let prog=data new%str(;) x=1%str(;)run%str(;);
```

Method Three: You could create an additional macro variable, assign a quoted value to it, and reference it in the assignment statement for the prog macro variable.

```
%let s=%str(;); %let prog=data new&s x=1&s run&s;
```

RangTech
Enhancing the way IT works

The %NRSTR Function

Sometimes you might want to hide the normal meaning of an ampersand or a percent sign. The %NRSTR function performs the same quoting function as %STR, except it also masks macro triggers (& and %). The **NR** in the name %NRSTR stands for **No Resolution**. %NRSTR has the same syntax as %STR.

Example

- Suppose you want to create a macro variable named period and to assign a value of *May&Jun* to it. If you attempt to use the %STR function in the assignment statement, SAS will interpret the ampersand as a macro trigger and generate a warning message. You need to use the %NRSTR function instead.

```
%let Period=%str(May&Jun);
%put Period resolves to: &period;
%let Period=%nrstr(May&Jun);
%put Period resolves to: &period;
```

RangTech
Enhancing the way IT works

Macro Character Functions

Often when working with macro variables, you will need to manipulate character strings. You can do this by using **macro character functions**.

With macro character functions, you can

- change lowercase letters to uppercase
- produce a substring of a character string
- extract a word from a character string
- determine the length of a character string, and more.

The %UPCASE Function

- The %UPCASE function enables you to change the value of a macro variable from lowercase to uppercase before substituting that value in a SAS program.

The %QUPCASE Function

- If the argument contains a special character, a mnemonic operator, or a macro trigger, you will need to use the %QUPCASE function. %QUPCASE has the same syntax as the %UPCASE function, and it works the same as %UPCASE except that it also masks mnemonic operators and special characters (including macro triggers).

Example

```
%let a=begin;   %let b=%nrstr(&a);
%put UPCASE produces: %upcase(&b); UPCASE produces: begin
%put QUPCASE produces: %qupcase(&b); QUPCASE produces: &A
```

RanaTech
Enhancing the way IT works

- **General form, %SUBSTR function:**

  %SUBSTR (*argument, position <,n>*)

where

  - *argument* is a character string or a text expression from which a substring will be returned.
  - *position* is an integer or an expression (text, logical, or mathematical) that yields an integer, which specifies the position of the first character in the substring.
  - *n* is an optional integer or an expression (text, logical, or mathematical) that yields an integer that specifies the number of characters in the substring.

For example, assume that the macro variable date has the value *01JAN2006*.

  - The code %substr(&date,3) will return the value *JAN2006*.
  - The code %substr(&date,3,3) will return the value *JAN*.
  - The code %substr(&date,3,9) will return the value *JAN2006* and will produce a warning message.

- The values of *position* and *n* can also be the result of a mathematical expression that yields an integer. For example, %substr(&var,%length(&var)-1) returns the last two characters of the value of the macro variable var.

- The **%LENGTH function** accepts an argument that is either a character string or a text expression. If the argument is a character string, %LENGTH returns the length of the string. If the argument is a text expression, %LENGTH returns the length of the resolved value. If the argument has a null value, %LENGTH returns 0.

The %QSUBSTR Function

- If the argument contains a special character, a mnemonic operator, or a macro trigger, you will need to use the %QSUBSTR function. %QSUBSTR has the same syntax as the %SUBSTR function, and it works the same as %SUBSTR except that it also masks mnemonic operators and special characters (including macro triggers).

Example

- These statements show the results produced by %SUBSTR and %QSUBSTR:

```
%let a=one;
%let b=two;
%let c=%nrstr(&a &b);

%put C: &c; C: &a &b

%put With SUBSTR: %substr(&c,1,2); With SUBSTR: one
%put With QSUBSTR: %qsubstr(&c,1,2); With QSUBSTR: &a
```

RangTech
Enhancing the way IT works

The %INDEX Function

- The %INDEX function enables you to determine the position of the first character of a string within another string.

%INDEX (*source*,*string*)

where both *source* and *string* are character strings or text expressions that can include
- constant text
- macro variable references
- macro functions
- macro calls.

The %INDEX function
- searches *source* for the first occurrence of *string*
- returns a number representing the position in *source* of the first character of *string* when there is an exact pattern match
- returns 0 when there is no pattern match.

Example

```
%let a=a very long value;
%let b=%index(&a,v);

%put V appears at position &b.;
```

V appears at position 3.

RangTech
Enhancing the way IT works

The %SCAN Function

- The %SCAN function enables you to extract words from the value of a macro variable.

%SCAN function: %SCAN (*argument* , *n* <,*delimiters*>)

where

- *argument* consists of constant text, macro variable references, macro functions, or macro calls.
- *n* is an integer or a text expression that yields an integer, which specifies the position of the word to return. If *n* is greater than the number of words in *argument*, the functions return a null string.
- *delimiters* specifies an optional list of one or more characters that separate "words" or text expressions that yield one or more characters.

Enhancing the way IT works

Example

```
%let libref=%scan(&syslast,1,.);
%let dsname=%scan(&syslast,2,.);
 proc datasets lib=&libref nolist;
 title "Contents of the Data Set &syslast";
 contents data=&dsname;
 run;    quit;
```

The %QSCAN Function

- If the argument contains a special character, a mnemonic operator, or a macro trigger, you will need to use the %QSCAN function. %QSCAN has the same syntax as the %SCAN function, and it works the same as %SCAN except that it also masks mnemonic operators and special characters (including macro triggers).

Example

```
%let a=one;
%let b=two;
%let c=%nrstr(&a*&b);
%put C: &c; C: &a*&b
%put With SCAN: %scan(&c,1,*); With SCAN: one
%put With QSCAN: %qscan(&c,1,*); With QSCAN: &a
```

RangTech
Enhancing the way IT works

Combining Macro Variable References with Text

- You can reference macro variables anywhere in your program. Some applications might require placing a macro variable reference adjacent to leading text (**text***&variable*) or trailing text (*&variable***text**) or referencing adjacent macro variables (*&variable****&variable***) in order to build a new token. When you combine macro variable references and text, it is important to keep in mind how SAS interprets tokens.

  Remember that a token ends when the word scanner detects either the beginning of a new token or a blank after a token.

  %let year=02; %let month=jan;
  proc chart data=sasuser.y**&year**&month; hbar week / sumvar=sale;
  run;

  %let lib=sasuser;
  libname **&lib** 'SAS-data-library';
  proc &graphics.chart data=**&lib..**y&year&month;
  Run;

RangTech
Enhancing the way IT works

- OPTIONS NOSYMBOLGEN | SYMBOLGEN;
- %PUT text ;
- %LET variable=value;
- %STR (argument)
- %NRSTR (argument)
- %UPCASE (argument)
- %QUPCASE (argument)
- %SUBSTR (argument, position <,n>)
- %QSUBSTR (argument, position <,n>)
- %INDEX (source,string)
- %SCAN (argument, n <,delimiters>)
- %QSCAN (argument, n <,delimiters>)
- %SYSFUNC (function(argument(s))<,format>)
- %QSYSFUNC (function(argument(s))<,format>)

RangTech
Enhancing the way IT works

Macro Variable At Execution

Introduction

- Because the macro facility performs its tasks before SAS programs execute, the information that the macro facility supplies does not depend on values that are accessed or computed during the execution of a SAS program. However, sometimes it is necessary to access or create macro variables during the execution of a SAS program. There are several methods that enable the macro facility to create or access macro variables **at execution time**. In this lesson, you learn to use macro variables during execution of
  - a DATA step
  - a PROC SQL step
  - an SCL program.

Objectives

- create macro variables during DATA step execution
- describe the difference between the SYMPUT routine and the %LET statement
- reference macro variables indirectly, using multiple ampersands for delayed resolution
- obtain the value of a macro variable during DATA step execution
- describe the difference between the SYMGET function and macro variable references
- create macro variables during PROC SQL execution
- store several values in one macro variable, using the SQL procedure
- create, update, and obtain the values of macro variables during the execution of an SCL program.

RangTech
Enhancing the way IT works

The SYMPUT Routine

- The DATA step provides functions and **CALL routines** that enable you to transfer information between an executing DATA step and the macro processor. You can use the SYMPUT routine to create a macro variable and to assign to that variable any value that is available in the DATA step.

CALL SYMPUT (*macro-variable*, *text*);

where

  - *macro-variable* is assigned the character value of *text*.
  - *macro-variable* and *text* can each be specified as
    - a literal, enclosed in quotation marks
    - a DATA step variable
    - a DATA step expression.

- When you use the SYMPUT routine to create a macro variable in a DATA step, the macro variable is not actually created and assigned a value until the DATA step is **executed**. Therefore, you cannot successfully reference a macro variable that is created with the SYMPUT routine by preceding its name with an ampersand within the same DATA step in which it is created.

RanaTech
Enhancing the way IT works

Using SYMPUT with a Literal

- In the SYMPUT routine, you use a literal string for
- the first argument to specify an **exact name** for the name of the macro variable
- the second argument to specify the **exact character value** to assign to the macro variable.
- To use a literal with the SYMPUT routine, you enclose the literal string in quotation marks.

CALL SYMPUT('*macro-variable*','*text*');

Example

```
options symbolgen pagesize=30; %let crsnum=3;
data revenue; set sasuser.all end=final;
if paidup<total then call symput('foot','Some Fees Are Unpaid'); run;
proc print data=revenue;
footnote "&foot"; run;
```

RangTech
Enhancing the way IT works

Using SYMPUT with a DATA Step Variable

CALL SYMPUT('*macro-variable*',*DATA-step-variable*);

- This form of the SYMPUT routine creates the macro variable named *macro-variable* and assigns to it the current value of *DATA-step-variable*.
- When you use a DATA step variable as the second argument,
  - a maximum of 32767 characters can be assigned to the receiving macro variable.
  - any leading or trailing blanks that are part of the DATA step variable's value are stored in the macro variable.
  - values of numeric variables are **automatically converted** to character values, using the BEST12. format.
  - If you enclose the DATA step variable name in quotation marks, SAS interprets the name as a **literal value** rather than as a variable name, and the DATA step variable's value is **not resolved**.

Example

```
data revenue;
set sasuser.all end=final;
if final then do;
call symput('numpaid',paidup);
call symput('numstu',total);
call symput('crsname',course_title); end; run;
proc print data=revenue noobs; title "Fee Status for &crsname (#&crsnum)";
footnote "Note: &numpaid Paid out of &numstu Students"; run;
```

CALL SYMPUT*('macro-variable',expression);*

A DATA step expression can be any combination of DATA step functions, DATA step variables, constants, and logical or arithmetic operators that resolves to a character or numeric constant.

- When you use a DATA step expression as the second argument, its current value is evaluated according to the following rules:
- Numeric expressions are **a**utomatically converted to character constants, using the BEST12. format.
- The resulting value can be up to 32767 characters long.
- Any leading or trailing blanks that are part of the expression are stored in the macro variable.

Example

data revenue;
set sasuser.all end=final; if paid='Y' then paidup+1; if final then do;
call symput('numpaid',trim(left(paidup)));
call symput('numstu',trim(left(total))); run;

CALL SYMPUT*(expression1,expression2)*

where

- *expression1* evaluates to a character value that is a valid macro variable name. This value should change each time you want to create another macro variable.
- *expression2* is the value that you want to assign to a specific macro variable.

data _null_;
set sasuser.courses; **call symput(course_code, trim(course_title));** run;
%put _user_;

RangTech
Enhancing the way IT works

Referencing Macro Variables Indirectly

The Forward Re-scan rule

The Forward Re-scan rule can be summarized as follows:

- When multiple ampersands or percent signs precede a name token, the macro processor resolves two ampersands (**&&**) to one ampersand (**&**), and re-scans the reference.
- To re-scan a reference, the macro processor scans and resolves tokens from left to right from the point where multiple ampersands or percent signs are coded, until no more triggers can be resolved.
- According to the Forward Re-scan rule, you need to use **three ampersands** in front of a macro variable name when its value matches the name of a second macro variable. This indirect reference resolves to the value of the second macro variable.

Reference	Scan	Resolved Value	Re-scan	Resolved Value
&crsid	→	C002	no re-scan	
&&crsid	→	&crsid	→	C002
&&&crsid	→	&C002	→	Structured Query Language

Romatech
Enhancing the way IT works

- You can create a series of macro variables, teach1 to teach*n*, each containing the name of the instructor who is assigned to a specific course.

```
options symbolgen;
data _null_;
set sasuser.schedule;
        call symput('teach'||left(course_number), trim(teacher));
   run;
```

Global Symbol Table	
TEACH1	*Hallis, Dr. George*
TEACH2	*Wickam, Dr. Alice*
TEACH3	*Forest, Mr. Peter*

```
%let crs=3;
proc print data=sasuser.register noobs;
title1 "Roster for Course &crs";
title2 "Taught by &&teach&crs";
run;
```

The SYMGET Function

Earlier you learned how to use the SYMPUT routine to create macro variables in a DATA step. You are also familiar with using a macro variable reference such as &macvar to obtain the value of a macro variable **before** a DATA step executes. Now, suppose you want to obtain the value of a macro variable **during** DATA step execution. You can obtain a macro variable's value during DATA step execution by using the SYMGET function. The SYMGET function returns the value of an existing macro variable.

SYMGET(*macro-variable*)

where *macro-variable* can be specified as one of the following:

- a macro variable name, enclosed in quotation marks
- a DATA step variable name whose value is the name of a macro variable
- a DATA step character expression whose value is the name of a macro variable.

Global Symbol Table	
TEACH1	*Hallis, Dr. George*
TEACH2	*Wickam, Dr. Alice*
TEACH3	*Forest, Mr. Peter*
CRS	*3*

data teachers; set sasuser.register; length Teacher $ 20;
teacher=symget('teach'||left(course_number)); run;

- **PROC SQL with the NOPRINT option and the INTO clause:**

PROC SQL NOPRINT;
SELECT *column1*<,*column2*,...> **INTO :***macro-variable-1*<, **:***macro-variable-2*,...>
FROM *table-1 | view-1*
<**WHERE** *expression*>'
<*other clauses*>**;**
QUIT;

where

- *column-1, column-2,...* specifies one or more columns of the SQL table specified by *table-1 | view-1*
- *:macro-variable-1, :macro-variable-2,...* names the macro variables to create
- *expression* produces a value that is used to subset the data
- *other clauses* are other valid clauses that group, subset or order the data.

Example

```
proc sql noprint;
select count(*) into :numrows from sasuser.schedule
where year(begin_date)=2002; %let numrows=&numrows;
%put There are &numrows courses in 2002;
select course_code, location, begin_date format=mmddyy10.
      into :crsid1-:crsid&numrows,
           :place1-:place&numrows,
from sasuser.schedule where year(begin_date)=2002 order by begin_date;
%put _user_; quit;
```

RangTech Changing the way IT works

General form, SELECT statement with INTO clause for combining values into one macro variable:

```
PROC SQL NOPRINT;
    SELECT column1  INTO :macro-variable-1    SEPARATED BY 'delimiter1'
        FROM table-1 | view-1
        <WHERE expression>
        <other clauses>;
QUIT;
```

Example

```
proc sql noprint;
select distinct location into :sites separated by ' '
from sasuser.schedule; quit;
```

Global Symbol Table	
Sites	*Boston Dallas Seattle*

RangTech
Enhancing the way IT works

Introduction

- Like macro variables, **macro programs (also known as macros)** enable you to **substitute text** into your SAS programs. Macros are different from macro variables because they can use conditional logic to make decisions about the text that you substitute into your programs. Using macros can help make your SAS programs more dynamic and reusable.

Objectives

- define and call simple macros
- describe the basic actions that the macro processor performs during macro compilation and execution
- use system options for macro debugging
- interpret error messages and warning messages that the macro processor generates
- define and call macros that include parameters
- describe the difference between positional parameters and keyword parameters
- explain the difference between the global symbol table and local symbol tables
- describe how the macro processor determines which symbol table to use
- describe the concept of nested macros and the hierarchy of symbol tables
- conditionally process code within a macro program
- iteratively process code within a macro program.

RangTech
Influencing the way IT works

General form, %MACRO statement and %MEND statement:

%MACRO *macro-name*;
text
%MEND <*macro-name*>;

where

- *macro-name* names the macro. The value of *macro-name* can be any valid SAS name that is not a reserved word in the SAS macro facility.
- *text* can be
  - constant text, possibly including SAS data set names, SAS variable names, or SAS statements
  - macro variables, macro functions, or macro program statements
  - any combination of the above.
- You might want to include *macro-name* in the %MEND statement in order to make your program more readable. However, the inclusion of *macro-name* in the %MEND statement is entirely optional.

Example

```
%macro prtlast;
 proc print data=&syslast (obs=5); title "Listing of &syslast data set"; run;
%mend;
```

RangTech
Enhancing the way IT works

Compiling a Macro

In order to use this macro later in your SAS programs, you must first **compile** it by submitting the macro definition, as follows:

```
%macro prtlast;
        proc print data=&syslast (obs=5);
        title "Listing of &syslast data set";
        run;
%mend;
```

When you submit this code, the word scanner divides the macro into tokens and sends the tokens to the macro processor for compilation.

The macro processor

- checks all macro language statements for syntax errors (non-macro language statements are not checked until the macro is executed).
- writes error messages to the SAS log and creates a **dummy (non-executable) macro** if any syntax errors are found in the macro language statements.
- stores all compiled macro language statements and constant text in a SAS catalog entry if no syntax errors are found in the macro language statements. By default, a catalog named **Work.Sasmacr** is opened, and a catalog entry named ***Macro-name*.Macro** is created.

RangTech
Enhancing the way IT works

Calling a Macro

- After the macro is successfully compiled, you can use it in your SAS programs for the duration of your SAS session without resubmitting the macro definition. Just as you must reference macro variables in order to access them in your code, you must call a macro program in order to execute it within your SAS program.

A **macro call**

- is specified by placing a percent sign (%) before the name of the macro
- can be made anywhere in a program **except** within the data lines of a DATALINES statement (similar to a macro variable reference)
- requires **no semicolon** because it is **not** a SAS statement.

To execute the macro **Prtlast** you would call the macro as follows:

```
proc sort data=sasuser.courses out=courses;
by course_code;
run;

%prtlast
```

RangTech
Enhancing the way IT works

Macro Execution

- When you call a macro in your SAS program, the word scanner passes the macro call to the macro processor, because the percent sign that precedes the macro name is a macro trigge**r**. When the macro processor receives *%macro-name*, it

1. searches the designated SAS catalog (**Work.Sasmacr** by default) for an entry named ***Macro-name*.Macro**.
2. executes compiled macro language statements within ***Macro-name***.
3. sends any remaining text in ***Macro-name*** to the input stack for word scanning.
4. suspends macro execution when the SAS compiler receives a global SAS statement or when it encounters a SAS step boundary.
5. resumes execution of macro language statements after the SAS code executes.

- remember that the macro call is processed by the macro processor **before** any SAS language statements such as DATA steps are compiled or executed. During macro execution, the macro processor can communicate directly with
  - both global and local symbol tables. For example, the macro processor can store macro variable values with a %LET statement and can resolve macro variable references.
  - the input stack. For example, the macro processor can generate SAS code for tokenization by the word scanner.

- During macro execution, the macro processor **cannot** communicate directly with SAS data sets in order to obtain or modify the values of DATA step variables.

Enhancing the way IT works

Example

- Let's look at an example of macro execution. Assume that the **Prtlast** macro has been compiled and that it has been stored in the **Work.Sasmacr** catalog.

1. First, you submit the macro call, as follows:
   %prtlast
2. When the word scanner encounters this call, it passes the call to the macro processor. The macro processor searches for the compiled macro in the catalog entry **Work.Sasmacr.Prtlast.Macro**.
   Catalog Entry
   %macro prtlast; proc print data=&syslast(obs=5); title "Listing of &syslast data set"; run; %mend;
3. The macro processor begins executing compiled macro language statements. However, in this example, no compiled macro statements are included in the macro.
4. The macro processor places noncompiled items (SAS language statements) on the input stack, and pauses as the word scanner tokenizes the inserted text. In this example, the macro processor will place the PROC PRINT step on the input stack. Input Stack proc print data=&syslast(obs=5); title "Listing of &syslast data set"; run;
5. The word scanner passes these tokens to the compiler. When the word scanner encounters a macro variable reference such as &syslast, it passes the reference to the macro processor for resolution. The macro processor returns the macro variable value to the input stack and word scanning continues.
6. After all of the statements in the PROC PRINT step have been compiled, the PROC PRINT step is executed, and SAS creates output that includes only the first five observations of the most recently created data set.
7. Once the PROC PRINT step has been executed, the macro processor resumes execution of any remaining macro language statements in the macro (there are none in this example). The macro processor ends execution when it reaches the %MEND statement.

Monitoring Execution with System Options

In the last example, you saw that when you call a macro, the text that is sent to the compiler does not appear in the SAS log. But sometimes you might want to see this text. There are SAS system options that can display information about the execution of a macro in the SAS log. This can be especially helpful for debugging purposes.

The MPRINT Option

OPTIONS MPRINT | NOMPRINT;

where

- **NOMPRINT** is the default setting, and specifies that the text that is sent to the compiler when a macro executes is not printed in the SAS log
- **MPRINT** specifies that the text that is sent to the compiler when a macro executes is printed in the SAS log.

```
options mprint; %prtlast
```

- The messages that are written to the SAS log show the text that is sent to the compiler. Notice that the macro variable reference (&SYSLAST) is resolved to the value *Work.Sales* in the MPRINT messages that are written to the SAS log.

```
%prtlast

MPRINT(PRTLAST): proc print data=WORK.SALES (obs=5);
MPRINT(PRTLAST): title "Listing of WORK.SALES";
MPRINT(PRTLAST): run;
```

RangTech
Enhancing the way IT works

The MLOGIC Option

OPTIONS MLOGIC | NOMLOGIC;

where

- **NOMLOGIC** is the default setting, and specifies that messages about macro actions are not printed to the SAS log during macro execution
- **MLOGIC** specifies that messages about macro actions are printed to the log during macro execution.

When the MLOGIC system option is in effect, the information that is displayed in SAS log messages includes

- the beginning of macro execution
- the results of arithmetic and logical macro operations
- the end of macro execution.

```
options nomprint mlogic; %prtlast;

%prtlast MLOGIC(PRTLAST): Beginning execution.
                          ..........
MLOGIC(PRTLAST): Ending execution.
```

The MLOGIC option, along with the MPRINT option and the SYMBOLGEN option, is typically turned

- **on** for development and debugging purposes
- **off** when the application is in production mode.

RangTech
Enhancing the way IT works

Overview:

```
%macro printdsn;
        proc print data=&dsn; var &vars; run;
%mend;
        %let dsn=sasuser.courses;
        %let vars=course_code course_title days;
        %printdsn
```

Macros That Include **Positional Parameters**

- When you include positional parameters in a macro definition, a macro variable is automatically created for each parameter when you call the macro. To define macros that include **positional parameters**, you list the names of macro variables in the %MACRO statement of the macro definition. Positional parameters are so named because the order in which you specify them in a macro definition determines the order in which they are assigned values from the macro call. That is, when you call a macro that includes positional parameters, you specify the values of the macro variables that are defined in the parameters in the **same order** in which they are defined.

```
%macro printdsn(dsn,vars);
        proc print data=&dsn; var &vars; title "Listing of %upcase(&dsn) data set";
        run;
 %mend;

%printdsn(sasuser.courses,course_code course_title days)
%printdsn(,course_code course_title days)
```

Macros That Include **Keyword Parameters**

- You can also include **keyword parameters** in a macro definition. Like positional parameters, keyword parameters create macro variables. However, when you use keyword parameters to create macro variables, you list both the name and the value of each macro variable in the macro definition.
- Keyword parameters can be listed in any order. Whatever value you assign to each parameter (or variable) in the %MACRO statement becomes its default value. Null values are allowed.

```
%macro printdsn(dsn=sasuser.courses,vars=course_code course_title days);
        proc print data=&dsn; var &vars; title "Listing of %upcase(&dsn) data set"; run;
%mend;

%printdsn(dsn=sasuser.schedule, vars=teacher course_code begin_date)
```

- To call the **Printdsn** macro with default values for the parameters
  - %printdsn()
  - %printdsn(dsn=sasuser.courses,vars=course_code course_title days)

Macros That Include **Mixed Parameter Lists**

- You can also include a parameter list that contains **both positional and keyword** parameters in your macro definitions. All positional parameter variables in the %MACRO statement must be listed **before** any keyword parameter variable is listed.

```
%macro printdsn(dsn, vars=course_title course_code days);
        proc print data=&dsn; var &vars; title "Listing of %upcase(&dsn) data set"; run;
%mend;
```

RangTech Enhancing the way IT works

Macros That Include the PARMBUFF Option

- You can use the PARMBUFF option in a macro definition to create a macro that can accept a **varying number of parameters** at each invocation. The PARMBUFF option assigns the entire list of parameter values in a macro call, including the parentheses in a name-style invocation, as the value of the automatic macro variable SYSPBUFF.

Example

- The following macro definition creates a macro named **Printz**. **Printz** uses a varying number of parameters and the automatic macro variable SYSPBUFF to display the parameters that are specified in the macro call. The macro also uses a conditional loop to print the data sets that are named as parameters, and the %EVAL function to increment the macro variable num. You will learn more about the %EVAL function later in this lesson.

```
%macro printz/parmbuff;
%put Syspbuff contains: &syspbuff;
%let num=1;
%let dsname=%scan(&syspbuff,&num);
%do %while(&dsname ne);
 proc print data=sasuser.&dsname; run;
%let num=%eval(&num+1);
%let dsname=%scan(&syspbuff,&num); %end;
%mend printz;
```

- If you submit a call to the macro that includes two parameters, the **Printz** macro writes the following line to the SAS log and causes two data sets to be printed:

```
%printz(courses, schedule)
```

Orangtech Enhancing the way IT works

The Global Symbol Table

- You are already somewhat familiar with the **global symbol table**. Remember that automatic macro variables are stored in the global symbol table. User-defined macro variables that you create with a %LET statement in open code (code that is outside of a macro definition) are also stored in the global symbol table. The global symbol table is created during the initialization of a SAS session and is deleted at the end of the session.

Macro variables in the global symbol table

- are available anytime during the session
- can be created by a user
- have values that can be changed during the session (except for some automatic macro variables).

Global Symbol Table	
SYSDATE	*01JUL02*
SYSDAY	*Monday*
SYSVER	*9.00*
uservar1	*value1*
uservar2	*value2*

You can create a global macro variable with

- a %LET statement (used outside a macro definition)
- a DATA step that contains a SYMPUT routine
- a SELECT statement that contains an INTO clause in PROC SQL
- a %GLOBAL statement.

RapaTech
Enhancing the way IT works

The %GLOBAL Statement

- creates one or more macro variables in the global symbol table and assigns null values to them
- can be used either inside **or** outside a macro definition
- has no effect on variables that are already in the global symbol table.

Example

- To create a global macro variable inside a macro definition, you can use the the %GLOBAL statement. The %GLOBAL statement in the following example creates two global macro variables, dsn and vars. The %LET statements assign values to the new global macro variables, as follows:

```
%macro printdsn;
        %global dsn vars;
        %let dsn=sasuser.courses;
        %let vars=course_title course_code days;
        proc print data=&dsn; var &vars; title "Listing of &dsn data set"; run;
%mend;
%printdsn
```

- You use the **%SYMDEL statement** to delete a macro variable from the global symbol table during a SAS session. To remove the macro variable dsn from the global symbol table, you submit the following statement:

```
%symdel dsn;
```

Rangtech
Enhancing the way IT works

The Local Symbol Table

- A **local symbol table** is created when a macro that includes a parameter list is called or when a request is made to create a local variable during macro execution. The local symbol table is deleted when the macro finishes execution. That is, the local symbol table exists only while the macro executes.

The local symbol table contains macro variables that can be 111
- created and initialized at macro invocation (that is, by parameters)
- created or updated during macro execution
- referenced anywhere within the macro.

You can create local macro variables with
- parameters in a macro definition
- a %LET statement within a macro definition
- a DATA step that contains a SYMPUT routine within a macro definition
- a SELECT statement that contains an INTO clause in PROC SQL within a macro definition
- a %LOCAL statement.

RangTech
Enhancing the way IT works

The %LOCAL Statement

- can appear **only** inside a macro definition
- creates one or more macro variables in the local symbol table and assigns null values to them
- has no effect on variables that are already in the local symbol table.
- A local symbol table is not created until a request is made to create a local variable.

- Macros that do not create local variables do not have a local table.
- The SYMPUT routine can create local variables only if the local table already exists.
- Since local symbol tables exist separately from the global symbol table, it is possible to have a local macro variable and a global macro variable that have the same name and different values.

Example

```
%let dsn=sasuser.courses;
%macro printdsn;
%local dsn;
%let dsn=sasuser.register;
%put The value of DSN inside Printdsn is &dsn;
%mend;
%printdsn %put The value of DSN outside Printdsn is &dsn;
```

- The value of DSN inside Printdsn is sasuser.register
- The value of DSN outside Printdsn is sasuser.courses

Rangtech Enhancing the way IT works

Rules for Creating and Updating Variables

Suppose the macro processor receives a %LET statement during a macro call, as follows:

%let macvar=value;

The macro processor will take the following steps:

Does macvar already exist in the local symbol table?

Yes → Update macvar in the local symbol table with the value value.

No ↓

Does macvar already exist in the global symbol table?

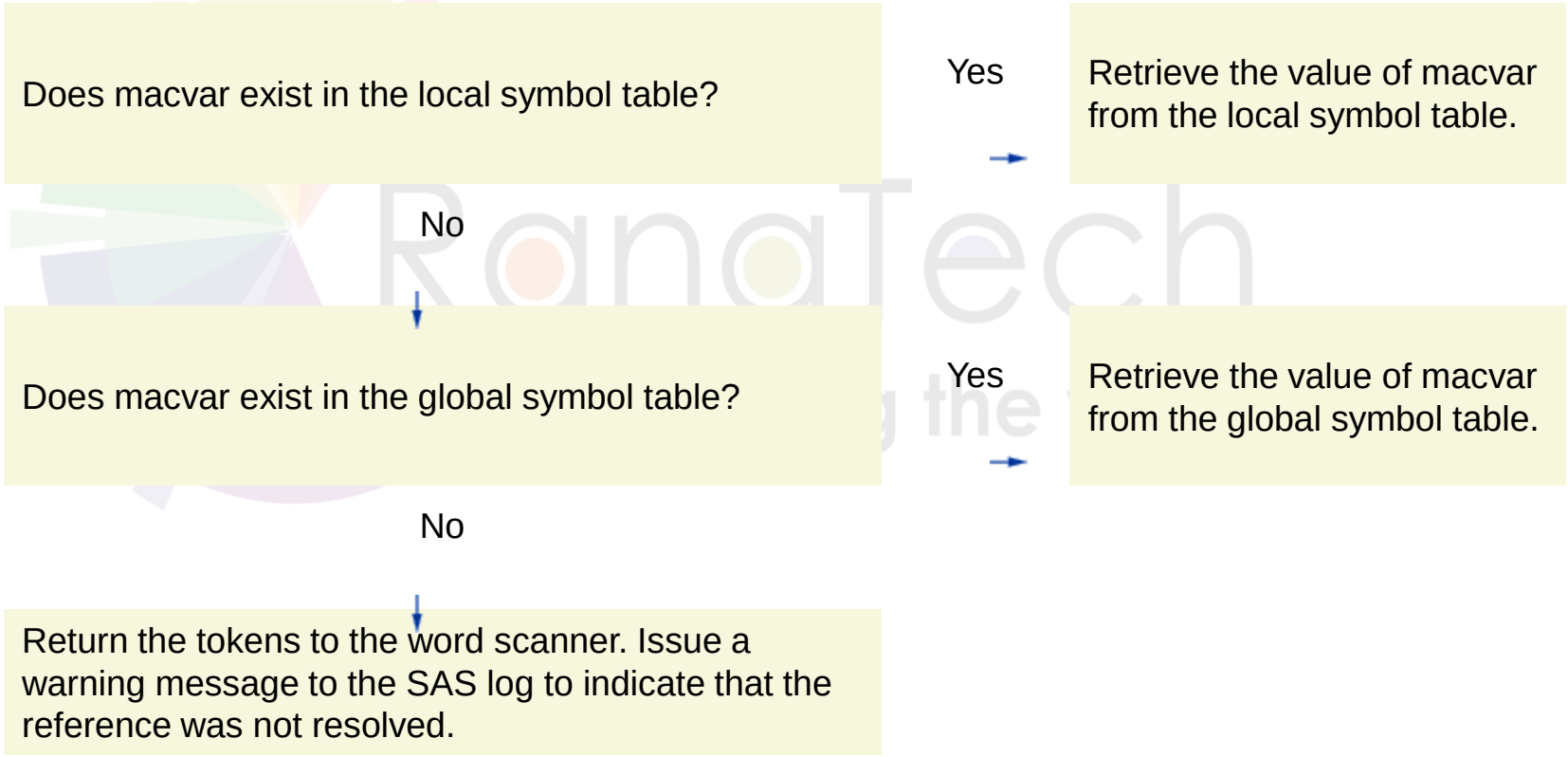
Yes → Update macvar in the global symbol table with the value value.

No ↓

Create macvar in the local symbol table and assign a value of value to it.

Symbol Tables..Con’t

- Similarly, suppose the macro processor receives the following reference during a macro call: &macvar



Multiple Local Symbol Tables

- Multiple local symbol tables can exist **concurrently** during macro execution if you have **nested macros**.

```
%macro outer;
                %local variX;
                %let variX=one;
        %inner
%mend outer;
%macro inner;
        %local variY;
        %let variY=&variX;
%mend inner;
```

Let's examine what happens to the symbol tables when you submit the following code:

```
%let variX=zero;
%outer
```

1.

Global Symbol Table	
variX	zero

2. The macro processor receives %outer. The macro processor retrieves the macro **Outer** from **Work.Sasmacr**, then begins executing it.

3. x

Global Symbol Table	
variX	zero

Outer Local Symbol Table	
variX	

RangTech
Enhancing the way IT works

4. The macro processor encounters %let variX=one;.

Global Symbol Table	
variX	*zero*

Outer Local Symbol Table	
variX	one

5. The macro processor receives %inner. It retrieves the macro **Inner** from **Work.Sasmacr**, then begins executing it.
6. The macro processor encounters %local variY;. It creates a local symbol table.

Global Symbol Table	
variX	*zero*

Outer Local Symbol Table	
variX	one

Inner Local Symbol Table	
variY	

7. The macro processor encounters %let variY=&variX;

Global Symbol Table	
variX	*zero*

Outer Local Symbol Table	
variX	one

Inner Local Symbol Table	
variY	*One*

8. The **Inner** macro finishes executing, and the local symbol table that was created within this macro is deleted.

Global Symbol Table	
variX	*zero*

Outer Local Symbol Table	
variX	one

9. The **Outer** macro finishes executing, and the local symbol table that was created within this macro is deleted. There are now no local symbol tables in existence. The global symbol table has not been changed since variX was created and was assigned a value of *zero*.

Global Symbol Table	
variX	*zero*

Processing Statements Conditionally : %IF ..%Then

```
%macro choice(status);
data fees;
set sasuser.all;
%if &status=PAID %then %do;
where paid='Y';
keep student_name course_code begin_date totalfee;
%end;
%else %do;
where paid='N';
keep student_name course_code begin_date totalfee latechg; latechg=fee*.10;
%end;
/* add local surcharge */
if location='Boston' then totalfee=fee*1.06;
else if location='Seattle' then totalfee=fee*1.025;
else if location='Dallas' then totalfee=fee*1.05;
run;
%mend choice;
```

RangTech
Enhancing the way IT works

Using Arithmetic and Logical Expressions

The %EVAL Function

The %EVAL function evaluates integer arithmetic or logical expressions.

- An arithmetic expression contains an arithmetic operator.
- A logical expression contains a logical operator.

The %EVAL function

- translates integer strings and hexadecimal strings to integers.
- translates tokens representing arithmetic, comparison, and logical operators to macro-level operators.
- performs arithmetic a

If you submit these statements...	These messages are written to the log...
%put value=%eval(10 lt 2);	value=0
%put value=10+2; %put value=%eval(10+2);	value=10+2 value=12
%let counter=2; %let counter=%eval(&counter+1); %put counter=&counter;	counter=3
%let numer=2; %let denom=8; %put value=%eval(&numer/&denom);	value=0
%let numer=2; %let demon=8; %put value=%eval(&numer/&denom*&denom); %put value=%eval(&denom*&numer/&denom);	value=0 value=2
%let real=2.4; %let int=8; %put value=%eval(&real+&int);	value=

General form, %SYSEVALF function:

%SYSEVALF(*expression<, conversion-type>***);** where

- *expression* is an arithmetic or logical expression to evaluate.
- *conversion-type* optionally converts the value returned by %SYSEVALF to the type of value specified. Conversion-type can be BOOLEAN, CEIL, FLOOR, or INTEGER.

The %SYSEVALF function evaluates arithmetic and logical expressions using floating-point arithmetic.

- ```
  %macro figureit(a,b);
  %let y=%sysevalf(&a+&b);
  %mend figureit;
  ```

- ```
  %let b=%sysevalf(10+20);
  %put 10+20 = &b;
  ```

RangTech
Enhancing the way IT works